HOSPITAL MANAGEMENT SYSTEM

Project Report

Object-Oriented Programming Course

Course Name:	**Object Oriented Programming**
Semester:	**BSCS 2-H**
Instructor:	**Sir Syed Muhammad Hassan/Sir Umer Amir**
University:	**SZABIST**
Submission Date:	**1/6/2026**

Group Members:

Kabir Karim	2512440
Shamoon Anjar Wala	2512458
Sufiyan Tariq	2512461

1. Introduction

Project Overview

The Hospital Management System (HMS), is a desktop application developed in Java using the Swing graphical user interface framework. The system is designed to digitize and streamline the core administrative operations of a hospital or medical clinic, replacing paper-based records with a fast, reliable, and user-friendly computer interface.

The application enables hospital staff to manage three core categories of data — Patients, Doctors, and Appointments — through a single unified interface. Each category supports full CRUD (Create, Read, Update, Delete) operations, and all data is persisted to local text files that survive application restarts.

Objectives

- Provide a complete patient management module with add, search, update, and delete
- Provide a complete doctor management module with the same operations
- Provide an appointment booking module with date/time validation and conflict detection
- Apply all four pillars of Object-Oriented Programming throughout the codebase
- Follow the Single Responsibility Principle (SRP) for every class
- Implement the MVC architecture separating UI from business logic
- Deliver a professional, polished user interface

2. Object-Oriented Design

The HMS project fully implements all four pillars of Object-Oriented Programming. This chapter explains how each pillar is applied with direct reference to the source code.

Encapsulation

All data fields in every model class are declared private. Data is accessed and modified exclusively through public getter and setter methods. This protects the internal state of objects and prevents unauthorized direct access.

```
class Patient extends Person {  18 usages

    private static int counter = 1000;  3 usages

    private int    age;  6 usages
    private String disease;  6 usages

    public int    getAge()     { return age; }  6 usages
    public String getDisease() { return disease; }  6 usages

    public void setAge(int age)            { this.age     = age; }  1 usage
    public void setDisease(String disease) { this.disease = disease; }  1 usage
```

The service classes also apply encapsulation — their internal ArrayList data structures are private, and external classes interact only through the public service methods.

```
import java.util.ArrayList;

class PatientService implements Service<Patient> {  3 usages

    private static final String    FILE     = "patients.txt";  3 usages
    private ArrayList<Patient>     patients = new ArrayList<>();  6 usages

    public PatientService() {  1 usage
        for (String line : FileUtil.read(FILE))
            patients.add(Patient.from(line));
    }
```

Inheritance

A Person abstract base class is the parent of both Patient and Doctor. Common attributes (id and name) are defined once in Person and inherited by both subclasses. This eliminates code duplication and establishes a clear class hierarchy.

```
abstract class Person {  2 usages  2 inheritors

    private int    id;  2 usages
    private String name;  3 usages

    // Constructor used by subclasses via super()
    public Person(int id, String name) {  4 usages
        this.id   = id;
        this.name = name;
    }

    // Common getters available to all subclasses
    public int    getId()   { return id; }  14 usages
    public String getName() { return name; }
    public void   setName(String name) { this.name = name; }  2 usages
```

Polymorphism

Person declares two abstract methods: toFile() and getSummary(). Both Patient and Doctor provide their own unique implementations of these methods. At runtime, Java automatically calls the correct version depending on the actual object type — this is runtime polymorphism via method overriding.

```
    @Override  4 usages
    public String toFile() { return getId() + "," + getName() + "," + age + "," + disease; }

    @Override  no usages
    public String getSummary() {
        return "Patient  | ID: " + getId()   +
               " | Name: "      + getName()  +
               " | Age: "       + age        +
               " | Disease: "   + disease;
    }

    public static Patient from(String s) {  1 usage
        String[] d = s.split( regex: ",",  limit: 4);
        return new Patient(
            Integer.parseInt(d[0].trim()),
            d[1].trim(),
            Integer.parseInt(d[2].trim()),
            d[3].trim()
        );
    }
}
```

Constructor overloading (compile-time polymorphism) is also used in all three model classes — one constructor for new records and one for loading from file.

Abstraction

Abstraction is implemented at two levels. First, the Person abstract class forces all subclasses to implement toFile() and getSummary() without specifying how. Second, the Service<T> generic interface defines a contract that all three service classes must fulfill.

```
abstract class Person {  2 usages  2 inheritors

    private int    id;  2 usages
    private String name;  3 usages

    // Constructor used by subclasses via super()
    public Person(int id, String name) {  4 usages
        this.id   = id;
        this.name = name;
    }

    // Common getters available to all subclasses
    public int    getId()   { return id; }  14 usages
    public String getName() { return name; }
    public void   setName(String name) { this.name = name; }  2 usages

    // Abstract method — every subclass MUST implement this
    // Pillar 4 (Abstraction): forces all Person subclasses to support file serialization
    public abstract String toFile();  4 usages  2 implementations

    // Abstract method — every subclass MUST implement this
    // Pillar 3 (Polymorphism): each subclass returns its own summary string
    public abstract String getSummary();  no usages  2 implementations
}
```

```java
import java.util.ArrayList;

interface Service<T> {  3 usages  3 implementations
    ArrayList<T> getAll();       // Return all records  8 usages  3 implementations
    T            findById(int id); // Find one by ID  16 usages  3 implementations
    boolean      delete(int id);   // Delete one by ID  3 usages  3 implementations
}
```

System Architecture

The project follows the Model-View-Controller (MVC) architectural pattern, which separates data, business logic, and UI into distinct layers with clear responsibilities.

Layer	Classes	Responsibility
Model	Patient, Doctor, Appointment, Person	Data structure and file serialization only
Service	PatientService, DoctorService, AppointmentService, AuthService	All business logic, validation, and file operations
View	LoginUI, HospitalUI	Display data and respond to user actions — no logic
Utility	FileUtil	Raw file read/write — no business knowledge
Interface	Service<T>	Abstraction contract for all service classes

Single Responsibility Principle (SRP)

Every class in the project has exactly one reason to change. The table below shows the single responsibility of each class.

Class	Single Responsibility
Person	Abstract base — shared person attributes
Patient	Patient data model only
Doctor	Doctor data model only
Appointment	Appointment data model only
Service<T>	Contract interface — defines what every service must offer
FileUtil	Raw file read/write — no business logic
PatientService	Patient CRUD operations and validation
DoctorService	Doctor CRUD operations and validation
AppointmentService	Appointment booking, validation, and conflict detection

Class	Single Responsibility
AuthService	Authentication — hashing and credential checking only
LoginUI	Login screen display and user input only
HospitalUI	Main window display and button event handling only

File Storage

Data is persisted in three plain-text CSV files in the application directory. Files are created automatically on first run if they do not exist.

Source Code

Person.java *Abstract base class — defines shared structure for Patient and Doctor.*

```
abstract class Person {

   private int    id;
   private String name;
   public Person(int id, String name) {
       this.id   = id;
       this.name = name;
   }
   public int    getId()   { return id; }
   public String getName() { return name; }
   public void   setName(String name) { this.name = name; }

   public abstract String toFile();

   public abstract String getSummary();
}
```

Patient.java *Extends Person. Adds age and disease fields. Overrides toFile() and getSummary().*

```
class Patient extends Person {
   private static int counter = 1000;

   private int    age;
   private String disease;
```

```java
public int    getAge()     { return age; }
public String getDisease() { return disease; }

public void setAge(int age)            { this.age     = age; }
public void setDisease(String disease) { this.disease = disease; }

public Patient(String name, int age, String disease) {
    super(++counter, name);
    this.age     = age;
    this.disease = disease;
}
public Patient(int id, String name, int age, String disease) {
    super(id, name);
    this.age     = age;
    this.disease = disease;

    if (id > counter) counter = id;
}
@Override
public String toFile() {
    return getId() + "," + getName() + "," + age + "," + disease;
}

@Override
public String getSummary() {
    return "Patient  | ID: " + getId()   +
           " | Name: "      + getName()  +
           " | Age: "       + age        +
           " | Disease: "   + disease;
}

public static Patient from(String s) {
    String[] d = s.split(",", 4);
    return new Patient(
        Integer.parseInt(d[0].trim()),
        d[1].trim(),
```

```
            Integer.parseInt(d[2].trim()),
            d[3].trim()
        );
    }
}
```

Doctor.java *Extends Person. Adds specialization field. Overrides toFile() and getSummary().*

```
class Doctor extends Person {

    private static int counter = 5000;
    private String spec;
    public Doctor(String name, String spec) {
        super(++counter, name);
        this.spec = spec;
    }

    public Doctor(int id, String name, String spec) {
        super(id, name);
        this.spec = spec;

        if (id > counter) counter = id;
    }

    public String getSpec() { return spec; }

    public void setSpec(String spec) { this.spec = spec; }
    @Override
    public String toFile() {
        return getId() + "," + getName() + "," + spec;
    }
    @Override
    public String getSummary() {
        return "Doctor   | ID: "           + getId()   +
               " | Name: "                 + getName() +
               " | Specialization: "       + spec;
    }
    public static Doctor from(String s) {
        String[] d = s.split(",", 3);
        return new Doctor(
            Integer.parseInt(d[0].trim()),
            d[1].trim(),
            d[2].trim()
        );
    }
}
```

Appointment.java *Standalone model — not a Person. Immutable once created (no setters).*

```
class Appointment {

   private static int counter = 9000;

   private int    id;
   private int    pid;
   private int    did;
   private String date;

   public Appointment(int pid, int did, String date) {
       this.id   = ++counter;
       this.pid  = pid;
       this.did  = did;
       this.date = date;
   }

   public Appointment(int id, int pid, int did, String date) {
       this.id   = id;
       this.pid  = pid;
       this.did  = did;
       this.date = date;

       if (id > counter) counter = id;
   }

   public int    getId()   { return id; }
   public int    getPid()  { return pid; }
   public int    getDid()  { return did; }
   public String getDate() { return date; }

   public String toFile() {
       return id + "," + pid + "," + did + "," + date;
   }

   public static Appointment from(String s) {
       String[] d = s.split(",", 4);
       return new Appointment(
           Integer.parseInt(d[0].trim()),
           Integer.parseInt(d[1].trim()),
           Integer.parseInt(d[2].trim()),
           d[3].trim()
       );
   }
}
```

Service.java *Generic interface — abstraction contract for all three service classes.*

```
import java.util.ArrayList;

interface Service<T> {
   ArrayList<T> getAll();       // Return all records
   T            findById(int id); // Find one by ID
   boolean      delete(int id);   // Delete one by ID
}
```

FileUtil.java *Utility class — handles raw file read, append, and overwrite. No business logic.*

```
import java.io.*;
import java.util.*;

class FileUtil {

   static void save(String file, String data) {
       try (FileWriter fw = new FileWriter(file, true)) {
           fw.write(data + "\n");
       } catch (Exception e) {
           System.err.println("[FileUtil] Error saving to " + file + ": " +
e.getMessage());
           e.printStackTrace();
       }
   }

   static ArrayList<String> read(String file) {
       ArrayList<String> list = new ArrayList<>();
       File f = new File(file);

       if (!f.exists()) {
           try { f.createNewFile(); }
           catch (Exception e) {
               System.err.println("[FileUtil] Could not create " + file + ": "
+ e.getMessage());
           }
           return list;
       }

       try (BufferedReader br = new BufferedReader(new FileReader(f))) {
           String l;
           while ((l = br.readLine()) != null) {
               if (!l.trim().isEmpty()) list.add(l);
           }
       } catch (Exception e) {
           System.err.println("[FileUtil] Error reading " + file + ": " +
e.getMessage());
           e.printStackTrace();
```

```
        }
        return list;
    }

    static void overwrite(String file, ArrayList<String> data) {
        try (FileWriter fw = new FileWriter(file)) {
            for (String line : data) fw.write(line + "\n");
        } catch (Exception e) {
            System.err.println("[FileUtil] Error overwriting " + file + ": " +
e.getMessage());
            e.printStackTrace();
        }
    }
}
```

AuthService.java *Handles authentication. Stores credentials as SHA-256 hash — no plain text, no file needed.*

```
import java.nio.charset.StandardCharsets;

import java.security.MessageDigest;



class AuthService {

    private static final String ADMIN_USER = "admin";

    private static final String ADMIN_HASH =

        "03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4";

    public boolean checkCredentials(String username, String password) {

        String hashedInput = hash(password);

        return ADMIN_USER.equals(username) && ADMIN_HASH.equals(hashedInput);

    }

    private String hash(String input) {

        try {

            MessageDigest md = MessageDigest.getInstance("SHA-256");
```

```
            byte[] bytes = md.digest(input.getBytes(StandardCharsets.UTF_8));

            StringBuilder sb = new StringBuilder();

            for (byte b : bytes) sb.append(String.format("%02x", b));

            return sb.toString();

        } catch (Exception e) {

            System.err.println("[AuthService] Hashing failed: " +
e.getMessage());

            e.printStackTrace();

            return "";

        }

    }

}
```

PatientService.java *Business logic for patients. Implements Service<Patient>. Handles add, update, delete, search, and validation.*

```
import java.util.ArrayList;

class PatientService implements Service<Patient> {

    private static final String    FILE     = "patients.txt";

    private ArrayList<Patient>     patients = new ArrayList<>();



    public PatientService() {

        for (String line : FileUtil.read(FILE))

            patients.add(Patient.from(line));

    }

    @Override

    public ArrayList<Patient> getAll() { return patients; }
```

```
@Override
public Patient findById(int id) {
    for (Patient p : patients)
        if (p.getId() == id) return p;
    return null;
}
@Override
public boolean delete(int id) {
    Patient p = findById(id);
    if (p == null) return false;
    patients.remove(p);
    saveAll();
    return true;
}
public String add(String name, String ageText, String disease) {
    if (name.isEmpty() || disease.isEmpty())
        return "Name and Disease cannot be empty.";
    int age;
    try { age = Integer.parseInt(ageText); }
    catch (NumberFormatException e) { return "Age must be a number."; }
    if (age <= 0 || age > 150)
        return "Please enter a valid age (1-150).";
    Patient p = new Patient(name, age, disease);
```

```
        patients.add(p);
        FileUtil.save(FILE, p.toFile());
        return null;
    }
    public String update(int id, String name, String ageText, String disease) {
        Patient p = findById(id);
        if (p == null) return "No patient found with ID " + id + ".";
        if (name.isEmpty() || disease.isEmpty())
            return "Name and Disease cannot be empty.";
        int age;
        try { age = Integer.parseInt(ageText); }
        catch (NumberFormatException e) { return "Age must be a number."; }
        if (age <= 0 || age > 150)
            return "Please enter a valid age (1-150).";
        p.setName(name);
        p.setAge(age);
        p.setDisease(disease);
        saveAll();
        return null;
    }
    private void saveAll() {
        ArrayList<String> lines = new ArrayList<>();
        for (Patient p : patients) lines.add(p.toFile());
        FileUtil.overwrite(FILE, lines);
```

```
    }

}
```

DoctorService.java *Business logic for Doctors. Implements Service<Patient>. Handles add, update, delete, search, and validation.*

```
import java.util.ArrayList;

class DoctorService implements Service<Doctor> {

    private static final String FILE    = "doctors.txt";
    private ArrayList<Doctor>   doctors = new ArrayList<>();

    public DoctorService() {
        for (String line : FileUtil.read(FILE))
            doctors.add(Doctor.from(line));
    }

    @Override
    public ArrayList<Doctor> getAll() { return doctors; }

    @Override
    public Doctor findById(int id) {
        for (Doctor d : doctors)
            if (d.getId() == id) return d;
        return null;
    }

    @Override
    public boolean delete(int id) {
        Doctor d = findById(id);
        if (d == null) return false;
        doctors.remove(d);
        saveAll();
        return true;
    }

    public String add(String name, String spec) {
        if (name.isEmpty() || spec.isEmpty())
            return "Name and Specialization cannot be empty.";

        Doctor d = new Doctor(name, spec);
        doctors.add(d);
        FileUtil.save(FILE, d.toFile());
        return null;
    }

    public String update(int id, String name, String spec) {
        Doctor d = findById(id);
```

```
        if (d == null) return "No doctor found with ID " + id + ".";
        if (name.isEmpty() || spec.isEmpty())
            return "Name and Specialization cannot be empty.";

        d.setName(name);
        d.setSpec(spec);
        saveAll();
        return null;
    }

    private void saveAll() {
        ArrayList<String> lines = new ArrayList<>();
        for (Doctor d : doctors) lines.add(d.toFile());
        FileUtil.overwrite(FILE, lines);
    }
}
```

HospitalUI.java

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.util.*;

import java.text.SimpleDateFormat;

import java.util.Date;



public class HospitalUI extends JFrame {

    private final PatientService     patientService     = new PatientService();

    private final DoctorService      doctorService      = new DoctorService();

    private final AppointmentService appointmentService = new
AppointmentService();

    private final DefaultTableModel pModel = new DefaultTableModel(

            new String[]{"ID", "Name", "Age", "Disease"}, 0);

    private final DefaultTableModel dModel = new DefaultTableModel(
```

```
            new String[]{"ID", "Name", "Specialization"}, 0);

    private final DefaultTableModel aModel = new DefaultTableModel(

            new String[]{"AppointmentID", "PatientID", "DoctorID", "DateTime"},
0);

    public HospitalUI() {

        setTitle("Hospital Management System");

        setSize(900, 580);

        setDefaultCloseOperation(EXIT_ON_CLOSE);



        JTabbedPane tabs = new JTabbedPane();

        tabs.add("Patients",     patientPanel());

        tabs.add("Doctors",      doctorPanel());

        tabs.add("Appointments", appointmentPanel());

        add(tabs);

        setVisible(true);

    }

    void addLabel(JPanel p, String text, int x, int y) {

        JLabel l = new JLabel(text);

        l.setBounds(x, y, 130, 25);

        p.add(l);

    }

    int askForId(String prompt) {

        String input = JOptionPane.showInputDialog(this, prompt);

        if (input == null || input.trim().isEmpty()) return -1;
```

```
        try {
            return Integer.parseInt(input.trim());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "ID must be a number.");
            return -1;
        }
    }
    boolean showIfError(String msg) {
        if (msg != null) {
            JOptionPane.showMessageDialog(this, msg);
            return true;
        }
        return false;
    }
    JPanel patientPanel() {
        JPanel p = new JPanel(null);

        JTextField name = new JTextField();
        JTextField age  = new JTextField();
        JTextField dis  = new JTextField();

        addLabel(p, "Name",     20, 20);  name.setBounds(150, 20,  150, 25);
p.add(name);
        addLabel(p, "Age",      20, 60);  age.setBounds(150,  60,  150, 25);
p.add(age);
```

```
        addLabel(p, "Disease", 20, 100); dis.setBounds(150,  100, 150, 25);
p.add(dis);



        JButton add    = new JButton("Add");

        JButton search = new JButton("Search");

        JButton delete = new JButton("Delete");

        JButton update = new JButton("Update");



        add.setBounds(20,  150, 90, 30);

        search.setBounds(120, 150, 90, 30);

        delete.setBounds(220, 150, 90, 30);

        update.setBounds(320, 150, 90, 30);



        p.add(add); p.add(search); p.add(delete); p.add(update);



        JTable table = new JTable(pModel);

        table.setAutoCreateRowSorter(true); // Sortable columns

        JScrollPane sp = new JScrollPane(table);

        sp.setBounds(20, 210, 840, 280);

        p.add(sp);



        // Populate table from service on load

        for (Patient pt : patientService.getAll()) {

            pModel.addRow(new Object[]{
```

```
                pt.getId(), pt.getName(), pt.getAge(), pt.getDisease()

            });

        }

        add.addActionListener(e -> {

            String err = patientService.add(

                name.getText().trim(),

                age.getText().trim(),

                dis.getText().trim()

            );

            if (showIfError(err)) return;



            Patient pt =
patientService.getAll().get(patientService.getAll().size() - 1);

            pModel.addRow(new Object[]{pt.getId(), pt.getName(), pt.getAge(),
pt.getDisease()});

            JOptionPane.showMessageDialog(this, "Patient Added. ID = " +
pt.getId());

            name.setText(""); age.setText(""); dis.setText("");

        });

        search.addActionListener(e -> {

            int id = askForId("Enter Patient ID to search:");

            if (id == -1) return;



            Patient pt = patientService.findById(id);

            if (pt == null) {
```

```
                JOptionPane.showMessageDialog(this, "No patient found with ID "
+ id + ".");

                return;

            }

            for (int i = 0; i < pModel.getRowCount(); i++) {

                if (Integer.parseInt(pModel.getValueAt(i, 0).toString()) == id)
{

                    table.setRowSelectionInterval(i, i);

                    table.scrollRectToVisible(table.getCellRect(i, 0, true));

                    break;

                }

            }



            name.setText(pt.getName());

            age.setText(String.valueOf(pt.getAge()));

            dis.setText(pt.getDisease());



            JOptionPane.showMessageDialog(this,

                "Patient found:\n" +

                "ID: "      + pt.getId()      + "\n" +

                "Name: "    + pt.getName()    + "\n" +

                "Age: "     + pt.getAge()     + "\n" +

                "Disease: " + pt.getDisease()

            );

        });
```

```
        delete.addActionListener(e -> {

            int id = askForId("Enter Patient ID to delete:");

            if (id == -1) return;



            Patient pt = patientService.findById(id);

            if (pt == null) {

                JOptionPane.showMessageDialog(this, "No patient found with ID "
+ id + ".");

                return;

            }



            int confirm = JOptionPane.showConfirmDialog(this,

                "Delete patient: " + pt.getName() + " (ID=" + id + ")?\n" +

                "This will also cancel all their appointments.",

                "Confirm Delete", JOptionPane.YES_NO_OPTION);

            if (confirm != JOptionPane.YES_OPTION) return;

            for (int i = 0; i < pModel.getRowCount(); i++) {

                if (Integer.parseInt(pModel.getValueAt(i, 0).toString()) == id)
{

                    pModel.removeRow(i); break;

                }

            }



            patientService.delete(id);

            appointmentService.deleteByPatient(id);
```

```
            for (int i = aModel.getRowCount() - 1; i >= 0; i--) {

                if (aModel.getValueAt(i,
1).toString().equals(String.valueOf(id)))

                    aModel.removeRow(i);

            }



            JOptionPane.showMessageDialog(this, "Patient and their appointments
deleted.");

        });

        update.addActionListener(e -> {

            int id = askForId("Enter Patient ID to update:");

            if (id == -1) return;



            Patient pt = patientService.findById(id);

            if (pt == null) {

                JOptionPane.showMessageDialog(this, "No patient found with ID "
+ id + ".");

                return;

            }



            JTextField newName = new JTextField(pt.getName());

            JTextField newAge  = new JTextField(String.valueOf(pt.getAge()));

            JTextField newDis  = new JTextField(pt.getDisease());



            JPanel form = new JPanel(new java.awt.GridLayout(3, 2, 5, 8));

            form.add(new JLabel("Name:"));    form.add(newName);
```

```
            form.add(new JLabel("Age:"));      form.add(newAge);
            form.add(new JLabel("Disease:")); form.add(newDis);

            int result = JOptionPane.showConfirmDialog(this, form,
                "Update Patient ID " + id, JOptionPane.OK_CANCEL_OPTION);
            if (result != JOptionPane.OK_OPTION) return;

            String err = patientService.update(
                id,
                newName.getText().trim(),
                newAge.getText().trim(),
                newDis.getText().trim()
            );
            if (showIfError(err)) return;
            for (int i = 0; i < pModel.getRowCount(); i++) {
                if (Integer.parseInt(pModel.getValueAt(i, 0).toString()) == id)
{
                    pModel.setValueAt(pt.getName(),    i, 1);
                    pModel.setValueAt(pt.getAge(),     i, 2);
                    pModel.setValueAt(pt.getDisease(), i, 3);
                    break;
                }
            }
```

```
            JOptionPane.showMessageDialog(this, "Patient ID " + id + " updated
successfully.");

        });



        return p;

    }

    JPanel doctorPanel() {

        JPanel p = new JPanel(null);



        JTextField name = new JTextField();

        JTextField spec = new JTextField();



        addLabel(p, "Name",           20, 20); name.setBounds(150, 20, 150,
25); p.add(name);

        addLabel(p, "Specialization", 20, 60); spec.setBounds(150, 60, 150,
25); p.add(spec);



        JButton add    = new JButton("Add");

        JButton search = new JButton("Search");

        JButton delete = new JButton("Delete");

        JButton update = new JButton("Update");



        add.setBounds(20,  120, 90, 30);

        search.setBounds(120, 120, 90, 30);

        delete.setBounds(220, 120, 90, 30);

        update.setBounds(320, 120, 90, 30);
```

```
        p.add(add); p.add(search); p.add(delete); p.add(update);



        JTable table = new JTable(dModel);

        table.setAutoCreateRowSorter(true);

        JScrollPane sp = new JScrollPane(table);

        sp.setBounds(20, 180, 840, 300);

        p.add(sp);

        for (Doctor d : doctorService.getAll()) {

            dModel.addRow(new Object[]{d.getId(), d.getName(), d.getSpec()});

        }

        add.addActionListener(e -> {

            String err = doctorService.add(

                name.getText().trim(),

                spec.getText().trim()

            );

            if (showIfError(err)) return;



            Doctor d = doctorService.getAll().get(doctorService.getAll().size()
- 1);

            dModel.addRow(new Object[]{d.getId(), d.getName(), d.getSpec()});

            JOptionPane.showMessageDialog(this, "Doctor Added. ID = " +
d.getId());

            name.setText(""); spec.setText("");

        });
```

```
        search.addActionListener(e -> {
            int id = askForId("Enter Doctor ID to search:");
            if (id == -1) return;

            Doctor d = doctorService.findById(id);
            if (d == null) {
                JOptionPane.showMessageDialog(this, "No doctor found with ID "
+ id + ".");
                return;
            }

            for (int i = 0; i < dModel.getRowCount(); i++) {
                if (Integer.parseInt(dModel.getValueAt(i, 0).toString()) == id)
{
                    table.setRowSelectionInterval(i, i);
                    table.scrollRectToVisible(table.getCellRect(i, 0, true));
                    break;
                }
            }

            name.setText(d.getName());
            spec.setText(d.getSpec());

            JOptionPane.showMessageDialog(this,
```

```
                "Doctor found:\n" +
                "ID: "             + d.getId()   + "\n" +
                "Name: "           + d.getName() + "\n" +
                "Specialization: " + d.getSpec()
            );
        });

        delete.addActionListener(e -> {
            int id = askForId("Enter Doctor ID to delete:");
            if (id == -1) return;

            Doctor d = doctorService.findById(id);
            if (d == null) {
                JOptionPane.showMessageDialog(this, "No doctor found with ID "
+ id + ".");
                return;
            }

            int confirm = JOptionPane.showConfirmDialog(this,
                "Delete doctor: " + d.getName() + " (ID=" + id + ")?\n" +
                "This will also cancel all their appointments.",
                "Confirm Delete", JOptionPane.YES_NO_OPTION);
            if (confirm != JOptionPane.YES_OPTION) return;
```

```
            for (int i = 0; i < dModel.getRowCount(); i++) {

                if (Integer.parseInt(dModel.getValueAt(i, 0).toString()) == id)
{

                    dModel.removeRow(i); break;

                }

            }



            doctorService.delete(id);

            appointmentService.deleteByDoctor(id);

            for (int i = aModel.getRowCount() - 1; i >= 0; i--) {

                if (aModel.getValueAt(i,
2).toString().equals(String.valueOf(id)))

                    aModel.removeRow(i);

            }



            JOptionPane.showMessageDialog(this, "Doctor and their appointments
deleted.");

        });

        update.addActionListener(e -> {

            int id = askForId("Enter Doctor ID to update:");

            if (id == -1) return;



            Doctor d = doctorService.findById(id);

            if (d == null) {

                JOptionPane.showMessageDialog(this, "No doctor found with ID "
+ id + ".");
```

```
                return;
            }

            JTextField newName = new JTextField(d.getName());
            JTextField newSpec = new JTextField(d.getSpec());

            JPanel form = new JPanel(new java.awt.GridLayout(2, 2, 5, 8));
            form.add(new JLabel("Name:"));           form.add(newName);
            form.add(new JLabel("Specialization:")); form.add(newSpec);

            int result = JOptionPane.showConfirmDialog(this, form,
                "Update Doctor ID " + id, JOptionPane.OK_CANCEL_OPTION);
            if (result != JOptionPane.OK_OPTION) return;

            String err = doctorService.update(
                id,
                newName.getText().trim(),
                newSpec.getText().trim()
            );
            if (showIfError(err)) return;

            for (int i = 0; i < dModel.getRowCount(); i++) {
                if (Integer.parseInt(dModel.getValueAt(i, 0).toString()) == id)
{
```

```
                    dModel.setValueAt(d.getName(), i, 1);

                    dModel.setValueAt(d.getSpec(), i, 2);

                    break;

                }

            }



            JOptionPane.showMessageDialog(this, "Doctor ID " + id + " updated
successfully.");

        });



        return p;

    }



    JPanel appointmentPanel() {

        JPanel p = new JPanel(null);



        JTextField pid = new JTextField();

        JTextField did = new JTextField();



        addLabel(p, "Patient ID",  20, 20);  pid.setBounds(150, 20,  150, 25);
p.add(pid);

        addLabel(p, "Doctor ID",   20, 60);  did.setBounds(150, 60,  150, 25);
p.add(did);

        addLabel(p, "Date & Time", 20, 100);



      java.util.Calendar cal = java.util.Calendar.getInstance();
```

```
        cal.set(java.util.Calendar.SECOND, 0);

        cal.set(java.util.Calendar.MILLISECOND, 0);

        Date minDate = cal.getTime();



        SpinnerDateModel model = new SpinnerDateModel(minDate, minDate, null,
java.util.Calendar.MINUTE);

        JSpinner dateSpinner = new JSpinner(model);

        JSpinner.DateEditor editor = new JSpinner.DateEditor(dateSpinner,
"yyyy-MM-dd HH:mm");

        dateSpinner.setEditor(editor);

        dateSpinner.setBounds(150, 100, 180, 25);

        p.add(dateSpinner);



        JButton book   = new JButton("Book");

        JButton search = new JButton("Search");

        JButton delete = new JButton("Delete");



        book.setBounds(20,   150, 90, 30);

        search.setBounds(120, 150, 90, 30);

        delete.setBounds(220, 150, 90, 30);



        p.add(book); p.add(search); p.add(delete);



        JTable table = new JTable(aModel);

        table.setAutoCreateRowSorter(true);
```

```
        JScrollPane sp = new JScrollPane(table);

        sp.setBounds(20, 210, 840, 280);

        p.add(sp);

        for (Appointment a : appointmentService.getAll()) {

            aModel.addRow(new Object[]{a.getId(), a.getPid(), a.getDid(),
a.getDate()});

        }



        book.addActionListener(e -> {

            Date selectedDate = (Date) dateSpinner.getValue();

            Date now          = new Date();

            if (selectedDate.before(now)) {

                JOptionPane.showMessageDialog(this,

                    "Cannot book an appointment in the past.\nPlease select a
future date and time.");

                java.util.Calendar resetCal = java.util.Calendar.getInstance();

                resetCal.set(java.util.Calendar.SECOND, 0);

                resetCal.set(java.util.Calendar.MILLISECOND, 0);

                dateSpinner.setValue(resetCal.getTime());

                return;

            }



            String selectedDateTime = new SimpleDateFormat("yyyy-MM-dd
HH:mm").format(selectedDate);



            String err = appointmentService.book(
```

```
                pid.getText().trim(),

                did.getText().trim(),

                selectedDateTime,

                patientService,

                doctorService

            );

            if (showIfError(err)) return;

            ArrayList<Appointment> all = appointmentService.getAll();

            Appointment a = all.get(all.size() - 1);

            aModel.addRow(new Object[]{a.getId(), a.getPid(), a.getDid(),
a.getDate()});

            JOptionPane.showMessageDialog(this, "Appointment Booked. ID = " +
a.getId());

            pid.setText(""); did.setText("");

        });

        search.addActionListener(e -> {

            int id = askForId("Enter Appointment ID to search:");

            if (id == -1) return;



            Appointment a = appointmentService.findById(id);

            if (a == null) {

                JOptionPane.showMessageDialog(this, "No appointment found with
ID " + id + ".");

                return;

            }
```

```
            for (int i = 0; i < aModel.getRowCount(); i++) {
                if (Integer.parseInt(aModel.getValueAt(i, 0).toString()) == id)
{
                    table.setRowSelectionInterval(i, i);
                    table.scrollRectToVisible(table.getCellRect(i, 0, true));
                    break;
                }
            }

            JOptionPane.showMessageDialog(this,
                "Appointment found:\n" +
                "Appointment ID: " + a.getId()   + "\n" +
                "Patient ID: "     + a.getPid()  + "\n" +
                "Doctor ID: "      + a.getDid()  + "\n" +
                "Date & Time: "    + a.getDate()
            );
        });

        delete.addActionListener(e -> {
            int id = askForId("Enter Appointment ID to delete:");
            if (id == -1) return;

            Appointment a = appointmentService.findById(id);
            if (a == null) {
```

```
                JOptionPane.showMessageDialog(this, "No appointment found with
ID " + id + ".");

                return;

            }



            int confirm = JOptionPane.showConfirmDialog(this,

                "Cancel appointment ID " + id + " on " + a.getDate() + "?",

                "Confirm Delete", JOptionPane.YES_NO_OPTION);

            if (confirm != JOptionPane.YES_OPTION) return;



            appointmentService.delete(id);



            for (int i = 0; i < aModel.getRowCount(); i++) {

                if (Integer.parseInt(aModel.getValueAt(i, 0).toString()) == id)
{

                    aModel.removeRow(i); break;

                }

            }



            JOptionPane.showMessageDialog(this, "Appointment deleted.");

        });



        return p;

    }

}
```

LoginUI.java

```
import javax.swing.*;

public class LoginUI extends JFrame {

   private final AuthService authService = new AuthService();

   JTextField     user;

   JPasswordField pass;

   public LoginUI() {

        setTitle("Login");

        setSize(300, 200);

        setLayout(null);

        setDefaultCloseOperation(EXIT_ON_CLOSE);



        JLabel u = new JLabel("Username:");

        u.setBounds(30, 30, 80, 25);

        add(u);



        user = new JTextField();

        user.setBounds(120, 30, 120, 25);

        add(user);



        JLabel p = new JLabel("Password:");

        p.setBounds(30, 70, 80, 25);

        add(p);
```

```
        pass = new JPasswordField();
        pass.setBounds(120, 70, 120, 25);
        add(pass);

        JButton login = new JButton("Login");
        login.setBounds(90, 110, 100, 30);
        add(login);

        login.addActionListener(e -> {
            String enteredUser = user.getText().trim();
            String enteredPass = new String(pass.getPassword());

            if (enteredUser.isEmpty() || enteredPass.isEmpty()) {
                JOptionPane.showMessageDialog(this,
                    "Username and password cannot be empty.");
                return;
            }

            if (authService.checkCredentials(enteredUser, enteredPass)) {
                new HospitalUI();
                dispose();
            } else {
                JOptionPane.showMessageDialog(this,
                    "Invalid username or password.");
```

```
            }

        });



        setVisible(true);

    }



    public static void main(String[] args) {

        new LoginUI();

    }

}
```

AppointmentService.java (key method) *Shows the doctor double-booking check — the most important validation in the system*

```
import java.util.ArrayList;

class AppointmentService implements Service<Appointment> {

    private static final String      FILE         = "appointments.txt";
    private ArrayList<Appointment>   appointments = new ArrayList<>();

    public AppointmentService() {
        for (String line : FileUtil.read(FILE))
            appointments.add(Appointment.from(line));
    }

    @Override
    public ArrayList<Appointment> getAll() { return appointments; }

    @Override
    public Appointment findById(int id) {
        for (Appointment a : appointments)
            if (a.getId() == id) return a;
        return null;
    }

    @Override
    public boolean delete(int id) {
        Appointment a = findById(id);
        if (a == null) return false;
```

```
        appointments.remove(a);
        saveAll();
        return true;
    }

    public String book(String pidText, String didText, String dateTime,
                       PatientService patientService, DoctorService
doctorService) {

        if (pidText.isEmpty() || didText.isEmpty())
            return "Patient ID and Doctor ID cannot be empty.";

        int pidVal, didVal;
        try {
            pidVal = Integer.parseInt(pidText);
            didVal = Integer.parseInt(didText);
        } catch (NumberFormatException e) {
            return "Patient ID and Doctor ID must be numbers.";
        }

        if (patientService.findById(pidVal) == null)
            return "Patient ID " + pidVal + " does not exist.";

        if (doctorService.findById(didVal) == null)
            return "Doctor ID " + didVal + " does not exist.";

        for (Appointment a : appointments) {
            if (a.getDid() == didVal && a.getDate().equals(dateTime)) {
                Doctor d = doctorService.findById(didVal);
                String docName = (d != null) ? d.getName() : "Doctor " + didVal;
                return "Dr. " + docName + " is already booked at " + dateTime +
                       ".\nPlease choose a different time slot.";
            }
        }

        for (Appointment a : appointments) {
            if (a.getPid() == pidVal && a.getDid() == didVal &&
a.getDate().equals(dateTime))
                return "This exact appointment already exists!";
        }

        Appointment a = new Appointment(pidVal, didVal, dateTime);
        appointments.add(a);
        FileUtil.save(FILE, a.toFile());
        return null;
    }

    public void deleteByPatient(int pid) {
        appointments.removeIf(a -> a.getPid() == pid);
```

```
        saveAll();
    }

    public void deleteByDoctor(int did) {
        appointments.removeIf(a -> a.getDid() == did);
        saveAll();
    }

    private void saveAll() {
        ArrayList<String> lines = new ArrayList<>();
        for (Appointment a : appointments) lines.add(a.toFile());
        FileUtil.overwrite(FILE, lines);
    }
}
```

User Manual

System Requirements

Requirement	Specification
Operating System	Windows 7 / 8 / 10 / 11 (or any OS with Java)
Java Version	Java 8 or higher (JDK required to compile, JRE to run)
RAM	Minimum 256 MB
Disk Space	Minimum 10 MB for application + data files
Display	Minimum 1024 x 768 resolution

Installation & Running

Step 1 — Place all files in one folder

Copy all 12 Java source files into a single folder on your computer:

```
Person.java          Doctor.java            DoctorService.java
Patient.java         Appointment.java        AppointmentService.java
Service.java         PatientService.java     AuthService.java
FileUtil.java        LoginUI.java            HospitalUI.java
```

Step 2 — Open Command Prompt in that folder

Right-click inside the folder while holding Shift → click 'Open command window here' (Windows), or use the cd command.

Step 3 — Compile

```
javac *.java
```

If successful, you will see no output and .class files will appear in the folder.

Step 4 — Run

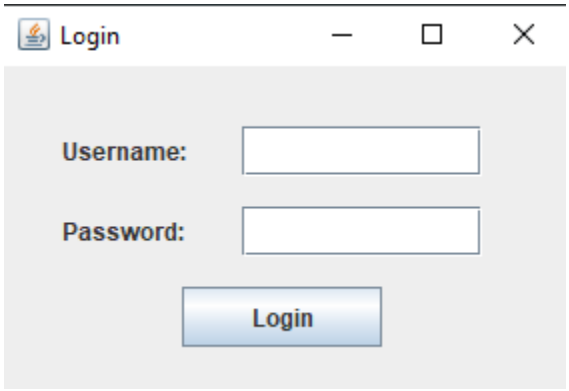
```
java LoginUI
```

The login window will appear on your screen.

Logging In

> **Default Credentials**
> Username: admin Password: 1234

- Enter admin in the Username field
- Enter 1234 in the Password field
- Click Sign In or press Enter
- If credentials are correct, the main HMS window opens
- If incorrect, an error message appears and the password field is cleared



Main Window Overview

After login, the HMS main window appears. It consists of two main areas:

Area	Description
Left Sidebar	Navigation panel with three sections: Patients, Doctors, Appointments. Click any to switch sections. Quick stats are shown at the bottom.
Main Content	Shows the active section with an input area at the top and a data table below.

Patient Management

Adding a Patient

- Navigate to the Patients section from the sidebar
- Enter the patient's Name in the Name field
- Enter the patient's Age (must be a number between 1 and 150)
- Enter the patient's Disease or condition
- Click the green Add button
- A success message shows the auto-generated Patient ID (starts from 1001)
- The new patient appears immediately in the table below

Searching for a Patient

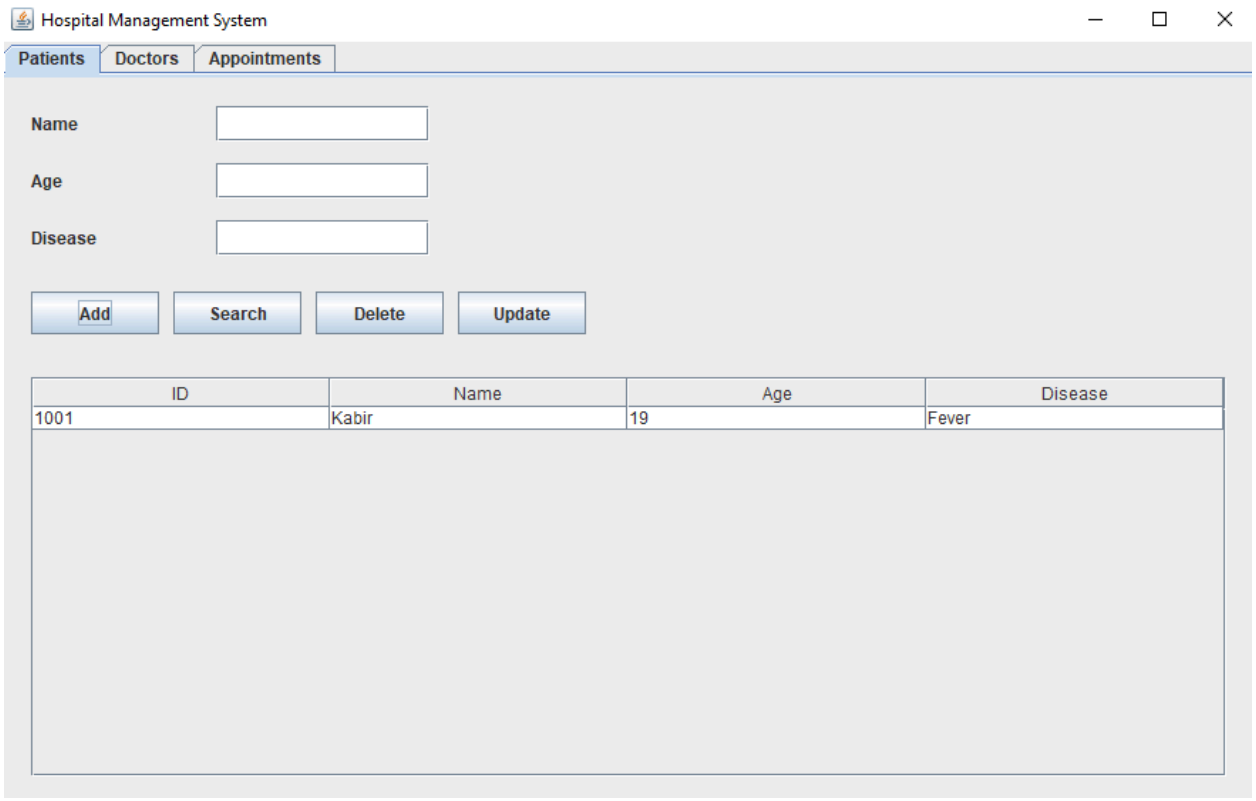
- Click the blue Search button
- A dialog box appears asking for a Patient ID
- Enter the ID and click OK
- If found: the row is highlighted in the table and details appear in a popup
- If not found: a message says 'No patient found with that ID'

Updating a Patient

- Click the purple Update button
- Enter the Patient ID to update
- A pre-filled dialog appears with the current Name, Age, and Disease
- Edit any fields you want to change
- Click OK to save — the table and file are updated instantly

Deleting a Patient

- Click the red Delete button
- Enter the Patient ID to delete
- A confirmation dialog shows the patient's name
- IMPORTANT: Deleting a patient also cancels all their appointments automatically
- Click Yes to confirm — the patient is removed from the table and file

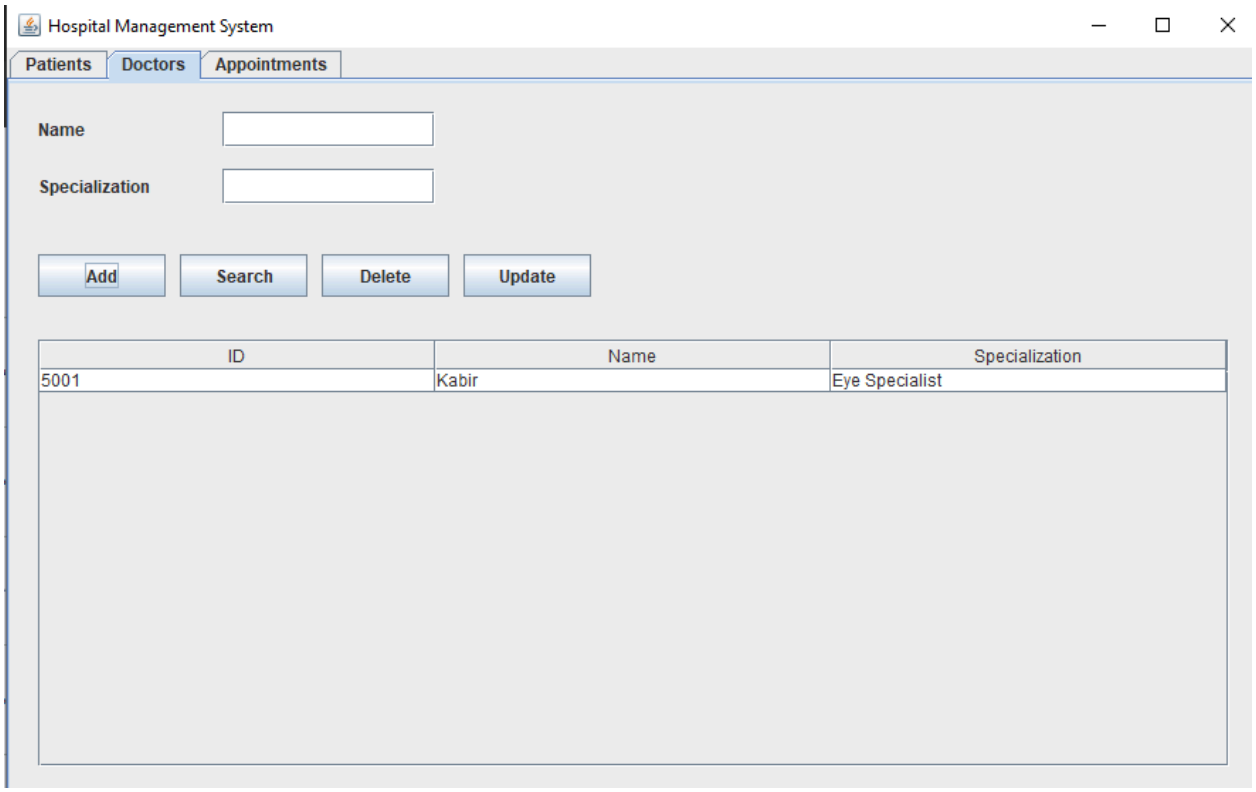


Doctor Management

The Doctor section works identically to the Patient section. The fields are:

- Name — the doctor's full name
- Specialization — the doctor's medical specialty (e.g. Cardiology, Pediatrics)

Doctor IDs are auto-generated starting from 5001. Deleting a doctor also removes all their appointments.

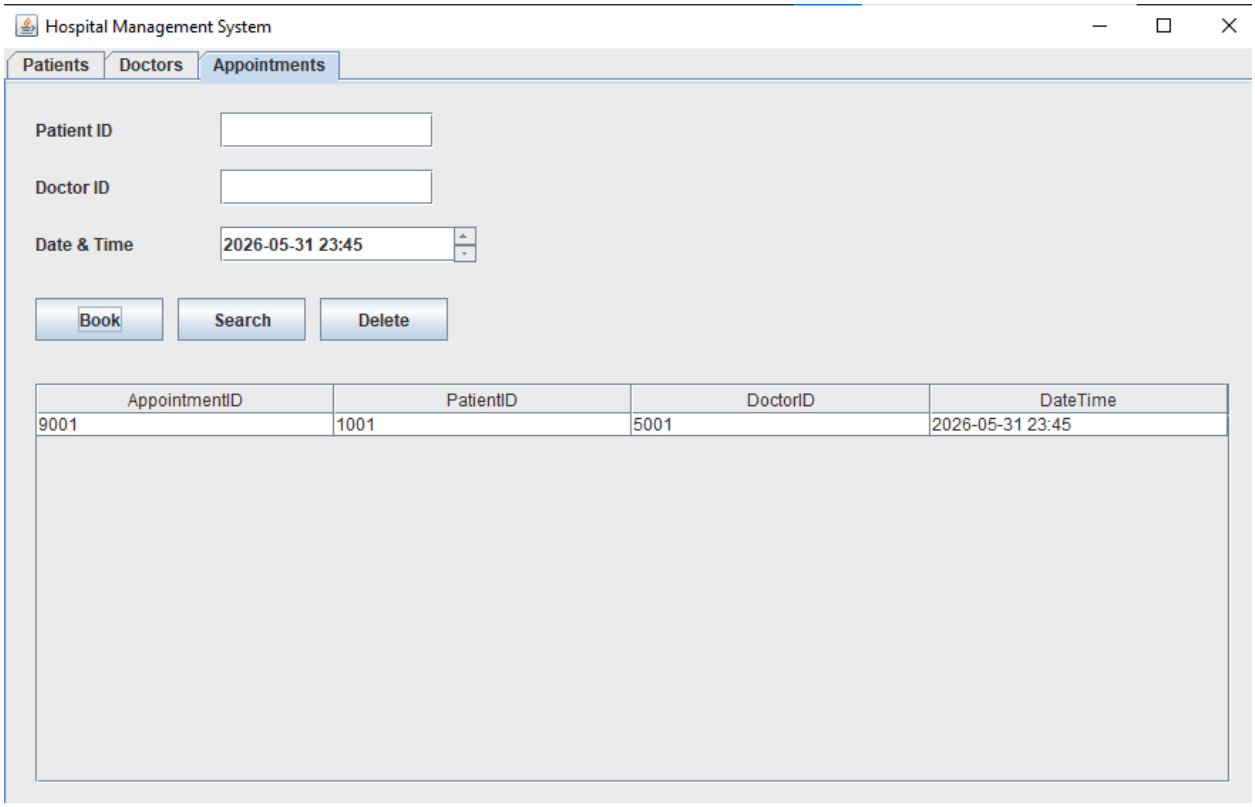


Appointment Booking

Booking an Appointment

- Navigate to the Appointments section
- Enter the Patient ID of the patient to book for
- Enter the Doctor ID of the doctor to book with
- Use the Date & Time spinner to select the appointment date and time
- The spinner only allows future dates — past dates are blocked

- Click the amber Book button



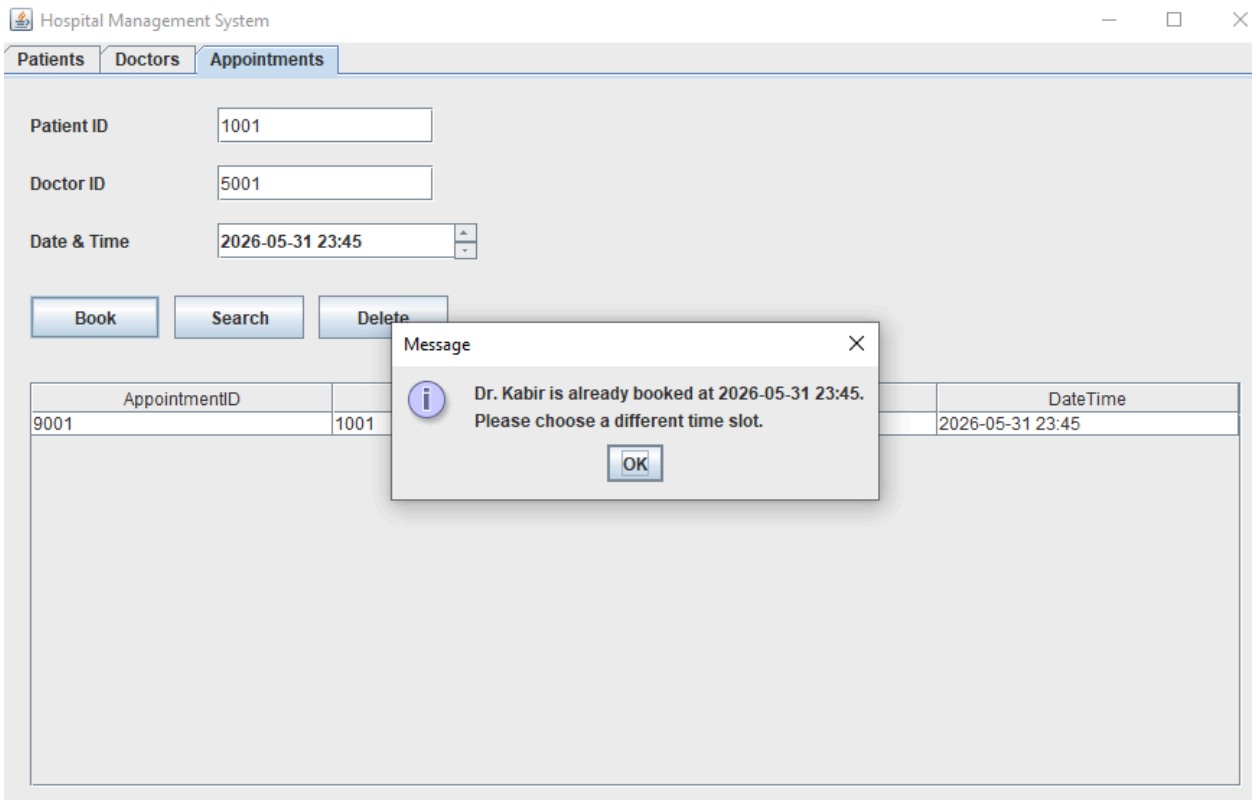
> **Doctor Double-Booking Prevention**
> If the selected doctor already has an appointment at the chosen time, the system will block the booking and show: "Dr. [Name] is already booked at [time]. Please choose a different time slot."

Searching an Appointment

- Click Search and enter the Appointment ID
- The appointment row is highlighted and details appear in a popup

Cancelling an Appointment

- Click the red Cancel button
- Enter the Appointment ID
- Confirm the cancellation — the appointment is removed



Data Storage

All data is automatically saved to files in the same folder as the application:

```
patients.txt      — all patient records
doctors.txt       — all doctor records
appointments.txt  — all appointment records
```

These files are created automatically on first run. Do not delete or manually edit them — doing so may cause the application to crash on the next load.

Error Messages Reference

Error Message	What to Do
Name and Disease cannot be empty	Fill in both the Name and Disease fields before clicking Add
Age must be a number	Enter only digits in the Age field (e.g. 35, not thirty-five)
Please enter a valid age (1-150)	Age must be between 1 and 150
Patient ID does not exist	Check the Patient ID — it must match an existing patient in the table
Doctor ID does not exist	Check the Doctor ID — it must match an existing doctor in the table
Dr. [Name] is already booked at [time]	Choose a different time slot for this doctor
This exact appointment already exists	This patient-doctor-time combination is already booked
Cannot book in the past	Select a future date and time using the date spinner
Invalid username or password	Check your credentials — default is admin / 1234

Testing

The following test cases verify all major features and edge cases of the application.

#	Test Case	Input	Expected Result	Status
1	Valid Login	admin / 1234	HMS window opens	PASS
2	Invalid Login	admin / wrong	Error dialog shown	PASS
3	Empty Login Fields	Empty username or password	Validation message shown	PASS
4	Add Patient	Name=Ali, Age=30, Disease=Fever	Patient added with auto ID	PASS
5	Add Patient — Bad Age	Name=Ali, Age=abc, Disease=Fever	Error: Age must be a number	PASS
6	Add Patient — Age > 150	Age=200	Error: valid age 1-150	PASS
7	Add Patient — Empty Name	Name empty	Error: cannot be empty	PASS
8	Search Patient — Valid	Existing Patient ID	Row highlighted, details shown	PASS
9	Search Patient — Invalid	Non-existent ID e.g. 9999	Error: not found	PASS
10	Update Patient	Valid ID, new data in dialog	Table and file updated	PASS
11	Delete Patient	Valid Patient ID	Patient removed, appointments cancelled	PASS
12	Add Doctor	Name=Dr. Sara, Spec=Cardiology	Doctor added with auto ID	PASS
13	Book Appointment	Valid Patient ID, Doctor ID, future dt	Appointment booked successfully	PASS
14	Double-Booking Check	Same Doctor ID at same time slot	Booking blocked with error	PASS
15	Past Date Booking	Date set to past time	Booking blocked: cannot book in past	PASS
16	Cancel Appointment	Valid Appointment ID	Appointment removed from table & file	PASS
17	Delete Doctor Cascade	Doctor with appointments	Doctor + appointments all deleted	PASS
18	Data Persistence	Add patient, restart app	Patient still appears after restart	PASS

Conclusion

The Hospital Management System successfully delivers a complete, functional desktop application that addresses the core administrative needs of a hospital. The system provides reliable patient, doctor, and appointment management with persistent file storage and a secure login mechanism.

From an Object-Oriented Programming perspective, the project demonstrates all four OOP pillars in a practical, real-world setting. Encapsulation protects data through private fields and accessor methods. Inheritance reduces code duplication through the Person base class. Polymorphism allows Patient and Doctor to have their own implementations of shared abstract methods. Abstraction is achieved through both an abstract class and a generic interface.

The Single Responsibility Principle is followed by all 12 classes — each class has exactly one reason to change. The MVC architecture clearly separates the UI layer from the business logic layer, making the code clean, maintainable, and easy to extend.

Features Delivered

- Secure login with SHA-256 hashed password
- Complete patient CRUD with input validation
- Complete doctor CRUD with input validation
- Appointment booking with doctor conflict detection
- Past-date booking prevention
- Cascade deletion (deleting a patient cancels their appointments)
- File persistence — data survives application restarts

OOP Pillars Summary

Pillar	Implementation	Classes Involved
Encapsulation	Private fields + getters/setters	Patient, Doctor, Appointment, all Services
Inheritance	extends Person base class	Patient extends Person, Doctor extends Person
Polymorphism	Override toFile() & getSummary(); constructor overloading	Patient, Doctor
Abstraction	abstract class Person + interface Service<T>	Person, Service, all 3 Services

Future Improvements

- Add a database backend (MySQL or SQLite) to replace text file storage
- Implement multiple user roles (Admin, Receptionist, Doctor)
- Add a billing and invoicing module
- Generate printable PDF reports for patient history
- Add email or SMS appointment reminders
- Implement a patient medical history log with multiple visit records